RAMOA – Task Breakdown

1. Event Normalization Engine (P-)

Before: 8–12 hrs
With AI: 5–7 hrs

What needs to be built:

- Unified event schema
- Raw log parser
- Event object generator

> Difficulty: Easy–Medium

Concepts required:

- JSON parsing
- Log structuring
- Python classes
- Data modeling

2. Heuristic Risk Module (P-)

Before: 6–8 hrs
With AI: 3–5 hrs
AI helps generate schema and parser quickly.

What needs to be built:

- Suspicious flag detection
- Weighted risk calculation
- Risk normalization between 0 and 1

> Difficulty: Easy

Concepts required:

- Feature engineering
- Weighted scoring
- Normalization

3. Graph Intelligence Layer (P-)

Before: 20–28 hrs
With AI: 14–18 hrs
Very fast with AI.

What needs to be built:

- Directed graph using NetworkX
- Node and edge schema
- Sliding time window pruning
- Degree deviation computation
- Rarity score computation
- Graph anomaly normalization

> Difficulty: Medium

Concepts required:

- Graph theory basics
- Centrality metrics
- NetworkX
- Sliding window logic
- Basic statistics

---

4. Risk Combination + Non-Linear Scaling (Praneeth)

Before: 4–6 hrs
With AI: 2–3 hrs
AI helps with:

- NetworkX boilerplate
- Metric calculation code
- Sliding window logic template

Still medium effort because debugging graph logic takes time.

What needs to be built:

- Combine heuristic risk and graph anomaly score
- Implement exponent scaling
- Ensure bounded output

> Difficulty: Easy

Concepts required:

- Weighted sums
- Exponentiation
- Risk modeling

---

5. RAMOA Orchestrator – Core Algorithm (Praneeth)

> Before: 20–25 hrs
> With AI: 15–18 hrs
> Very small.

What needs to be built:

- Expected Utility formula
- Agent selection logic
- Threshold mechanism
- Diversity safeguard rule
- Random sampling safeguard
- Escalation override rule

> Difficulty: Medium–Hard

Concepts required:

- Decision theory
- Cost-sensitive logic
- Control flow architecture
- Defensive system design

This is the core of the project and must remain under your control.

---

6. Trust Score Engine (Praneeth)

> Before: 12–16 hrs
> With AI: 8–12 hrs
> AI reduces boilerplate and update math coding.

What needs to be built:

- Trust initialization
- Positive update rule
- Negative update rule

- Trust decay mechanism
- Upper and lower bounds
- Stabilization smoothing

> Difficulty: Medium

Concepts required:

- Reinforcement-style updating
- Learning rate logic
- Bounded numerical systems
- Stability handling

---

7. Analytical Agents (Harsha)

> Before: 20–30 hrs
> With AI: 14–20 hrs
> AI speeds up:
>
> - Isolation Forest setup
> - Rule scaffolding
> - Interface design
>
> But tuning still manual.

What needs to be built:

- Process heuristic agent
- Network behavior agent
- Statistical anomaly agent
- Standard agent interface

Each agent must:

- Take event as input
- Return verdict
- Return confidence
- Return evidence

> Difficulty: Medium

Concepts required:

- Rule-based detection
- Scikit-learn basics
- Anomaly detection

- Clean modular design

---

8. Confidence Fusion Layer (Praneeth)

> Before: 4–6 hrs
> With AI: 2–3 hrs
> Very fast.

What needs to be built:

- Weighted aggregation of agent outputs
- Final confidence calculation
- Escalation threshold logic

> Difficulty: Easy

Concepts required:

- Weighted averaging
- Threshold systems
- Aggregation logic

---

9. Knowledge Base – Exact-Match Cache (Harsha)

> Before: 12–16 hrs
> With AI: 8–10 hrs
> AI generates hashing + TTL templates quickly.

What needs to be built:

- Event signature generator
- Hashing mechanism
- TTL expiration logic
- Cache lookup
- Verified result storage
- Revalidation trigger

> Difficulty: Medium

Concepts required:

- Hashing
- Dictionary or SQLite
- Time-based expiration

- Cache logic

---

10. Evaluation Framework (Koushal/Praneeth)

> Before: 25–35 hrs
> With AI: 18–24 hrs
> AI helps with:
>
> - Confusion matrix
> - Precision/Recall
> - Logging template
> - Plot generation
>
> But experiment design still yours.

What needs to be built:

- Static baseline mode
- RAMOA selective mode
- Metric logging system
- Latency measurement
- Agent invocation counter
- Precision, recall, F1 computation
- Graph vs no-graph comparison
- Cache efficiency analysis

> Difficulty: Medium–Hard

Concepts required:

- Experimental design
- Confusion matrix
- Performance metrics
- Controlled comparisons
- Data logging and analysis

You must control evaluation because it defines the paper and proof of novelty.

---

11. System Integration Layer (Praneeth)

Before: 15–20 hrs
With AI: 12–15 hrs
AI helps debug wiring issues.
Still unpredictable.

What needs to be built:

- Execution pipeline
- Module communication control
- Clean control flow
- Logging system
- Testing harness

> Difficulty: Medium

Concepts required:

- Modular software architecture
- Interface contracts
- Debugging discipline
- Clean integration design

You own final integration to prevent architectural chaos.

---

12. Documentation and Paper Preparation (P 1234)

What needs to be built:

- Literature review
- Architecture diagrams
- Mathematical formulation section
- Experimental documentation
- Formatting and structuring

> Difficulty: Medium

Concepts required:

- Technical writing
- Research structure
- Comparative analysis

Final Responsibility Summary